Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Южно-Уральский государственный университет»
(национальный исследовательский университет)
Институт естественных и точных наук
Кафедра прикладной математики и программирования

Тема: Пекарня-кондитерская

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ
по дисциплине администрирование и проектирование хранилищ данных
ЮУрГУ–010302.2023.063.ПЗ КР

Руководитель работы,
_____/Т.Ю. Оленчикова
«____»_____2025 г.

Автор работы
Студент группы ЕТ-313
_____/ М.М. Седова
«____»_____2025 г.

Челябинск 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
1 ПОСТАНОВКА ЗАДАЧИ 4
2 КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ 6
3 ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ 11
4 ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ 19
5 ОБЕСПЕЧЕНИЕ ЦЕЛОСТНОСТИ 21
6 ОБЕСПЕЧЕНИЕ БЕЗОПАСНОСТИ 23
7 ОПИСАНИЕ ПРИЛОЖЕНИЯ 25
8 РУКОВОДСТВО ПО РАБОТЕ С ПРОГРАММОЙ 27
ЗАКЛЮЧЕНИЕ 28
ЛИТЕРАТУРА 29
ПРИЛОЖЕНИЕ 1. Создание Базы данных 30
ПРИЛОЖЕНИЕ 2. Код программы 39

ВВЕДЕНИЕ

В условиях растущей конкуренции и увеличения объемов продаж эффективное управление бизнесом невозможно без автоматизации учетных процессов. Пекарня-кондитерская, занимающаяся производством и продажей хлебобулочных и кондитерских изделий, – не исключение и также нуждается в надежной и гибкой информационной системе.

Цель данной курсовой работы – проектирование и разработка реляционной базы данных и создание клиентского консольного приложения для работы с системой. В работе реализованы ключевые бизнес-процессы: регистрация клиентов, оформление заказов, управление ассортиментом, формирование отчетов, контроль остатков в филиалах и просмотр продукции.

Задачи, решаемые в рамках работы:

- анализ предметной области;
- разработка концептуальной, логической и физической модели БД;
- реализация базы данных в СУБД PostgreSQL;
- обеспечение целостности и безопасности данных;
- разработка приложения на языке Python для взаимодействия с БД.

1 ПОСТАНОВКА ЗАДАЧИ

Пекарня-кондитерская занимается производством и продажей кондитерских изделий, выпечки и хлеба. Основная деятельность включает розничные продажи в собственных точках и выполнение заказов.

Продукция изготавливается из ингредиентов, которые закупаются у поставщиков. Каждое изделие имеет состав и цену. Продажи осуществляются как в розницу, так и по предварительным заказам. Для производства и продажи привлекаются сотрудники: пекари, кондитеры, продавцы, менеджеры и администраторы. Каждый сотрудник работает по графику и получает заработную плату.

Отчеты:

- Продажи по дням, неделям, месяцам.
- Самые популярные товары.
- История заказов клиента.
- Использование ингредиентов.
- Финансовый оборот.

Организация ведет учет:

- клиентов (ФИО, номер телефона);
- заказов (дата, сумма, клиент, сотрудник);
- состава заказов (товары и их количество);
- продукции (название, категория, цена);
- состава продукции (ингредиенты и их количество);
- ингредиентов (название);
- сотрудников (ФИО, должность, дата найма, филиал);
- должностей (название);
- филиалов (адрес);
- остатков продукции в филиале (товары и их количество, филиал).

Цель работы – создание базы данных для автоматизации учета продукции, заказов, продаж, персонала и финансовых операций пекарни-кондитерской.

Пользователи системы (роли):

1. Клиент (К) – просматривает ассортимент и совершает покупки.
2. Продавец (П) – оформляет заказы и регистрирует клиентов.
3. Менеджер (М) – управляет ассортиментом, ценами, персоналом.
4. Администратор (А) – настраивает систему, формирует отчеты, контролирует данные.

Автоматизируемые бизнес-процессы:

1. Просмотр доступного ассортимента (К).

2. Регистрация клиента и его контактных данных (П).
3. Оформление заказа (П).
4. Управление ассортиментом: добавление, изменение, удаление продукции (М).
5. Ведение состава продукции (М).
6. Учет продаж и формирование отчетов (А).

Сущности:

- покупатели;
- сотрудники;
- должности (сотрудников);
- заказы;
- составы заказов;
- товары;
- категории (товаров);
- составы товаров;
- ингредиенты;
- точки фирмы (филиалы);
- товары в точке.

2 КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ

На этапе концептуального проектирования была разработана информационная модель предметной области – ER-диаграмма (рисунок 1), которая отражает основные объекты и связи между ними без привязки к СУБД. Модель построена на основе технического задания и бизнес-процессов пекарни-кондитерской.

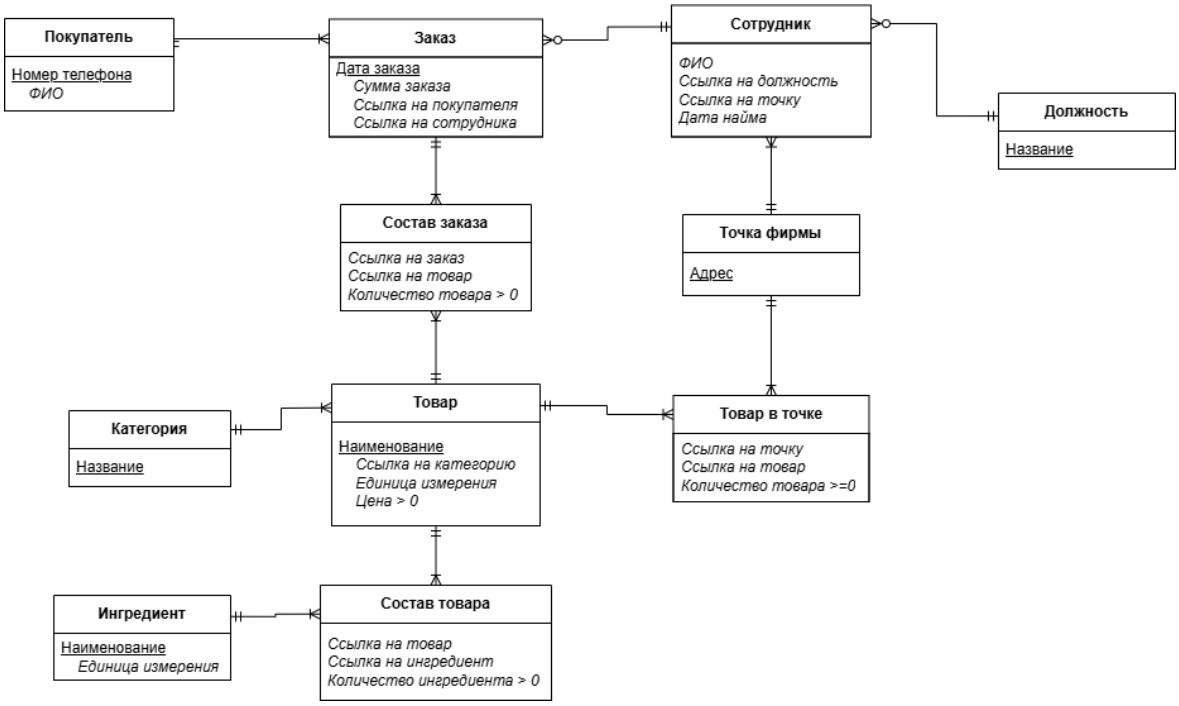

Рисунок 1 – ER-диаграмма

В модели выделены следующие ключевые сущности: Покупатель, Сотрудник, Заказ, Товар, Ингредиент, Категория, Должность, Филиал. Между ними установлены связи «один ко многим» и «многие ко многим», реализованные через внешние ключи и промежуточные таблицы («Состав заказа», «Состав товара»).

В таблице 1 представлены атрибуты каждой сущности, включая их тип данных, обязательность, уникальность и ограничения.

Таблица 1 –Таблица атрибутов

Сущность	Наименование атрибута	Описание	Тип данных	Значение по умолчанию	Обязательность	Уникальность	Ограничения
Покупатель	Номер телефона	Номер телефона	текст	-	+	+	Формат: 79999999999
	ФИО	Фамилия Имя Отчество	текст	-	+	-	-
Заказ	Ссылка на сотрудника	Продавец (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Ссылка на покупателя	Покупатель, совершивший заказ (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Сумма	Сумма заказа (сумма товаров из состава заказа)	DECIMAL(10, 2) (дробь)	-	+	-	> 0
	Дата	Дата заказа	date	-	+	-	Формат: ГГГГ-ММ-ДД

Категория	Название	Название категории	текст	-	+	+	-
Товар	Наимено-вание	Название товара	текст	-	+	+	-
	Ссылка на категорию	Категория товара (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Единица измере-ния	Единица измерения товара	текст	-	+	-	Формат: шт, гр, кг
	Цена	Цена товара	DECIMAL(10, 2) (дробь)	-	+	-	> 0
Состав заказа	Ссылка на заказ	Заказ (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Ссылка на товар	Товар из заказа (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Количес-тво товара	Количество проданного товара	DECIMAL(10, 2) (дробь)	-	+	-	(0; Товар в точке.количество]

Ингредиент	Наимено-вание	Название ингредиента	текст	-	+	+	-
	Единица измере-ния	Единица измерения	текст	-	+	-	Формат: шт, гр
Состав товара	Ссылка на нгреди-ент	Ингредиент из состава товара (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Количес-тво ин-гредиента	Количество ингредиента	DECIMAL(10, 2) (дробь)	-	+	-	> 0
	Ссылка на товар	Товар (FK, берем из другой таблицы)	INT (число)	-	+	-	-
Должность	Название	Название должности	текст	-	+	+	-
Сотрудник	ФИО	Фамилия Имя Отчество	текст	-	+	-	-
	Ссылка на долж-ность	Должность сотрудника (FK, берем из другой таблицы)	INT (число)	-	+	-	-

	Ссылка на точку	Филиал сотрудника (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Дата найма	Дата начала работы сотрудника	date	-	+	-	Формат: ГГГГ-ММ-ДД
Точка	Адрес	Адрес точки (филиала)	текст	-	+	+	-
Товар в точке	Ссылка на точку	Точка (филиал), в которой находится товар (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Ссылка на товар	Товар (FK, берем из другой таблицы)	INT (число)	-	+	-	-
	Количес-тво товара	Оставшееся количество товара	DECIMAL(10, 2) (дробь)	-	+	-	>= 0

Эта модель служит основой для последующих этапов проектирования – логического и физического.

3 ЛОГИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

На этапе логического проектирования концептуальная модель была преобразована в реляционную модель, представляющую собой набор таблиц, связанных между собой внешними ключами. Модель полностью соответствует требованиям нормализации (до 3НФ) и учитывает все бизнес-правила, сформулированные в техническом задании.

В реляционной модели (рисунок 2) выделено 11 таблиц, каждая из которых соответствует одной из сущностей из концептуальной модели:

- Customer – клиенты;
- Employee – сотрудники;
- Position – должности;
- Branch – филиалы (точки);
- Category – категории товаров;
- Product – товары;
- Ingredient – ингредиенты;
- Product_composition – состав товаров;
- Branch_product – остатки товаров в филиалах;
- "Order" – заказы;
- Order_composition – состав заказов.

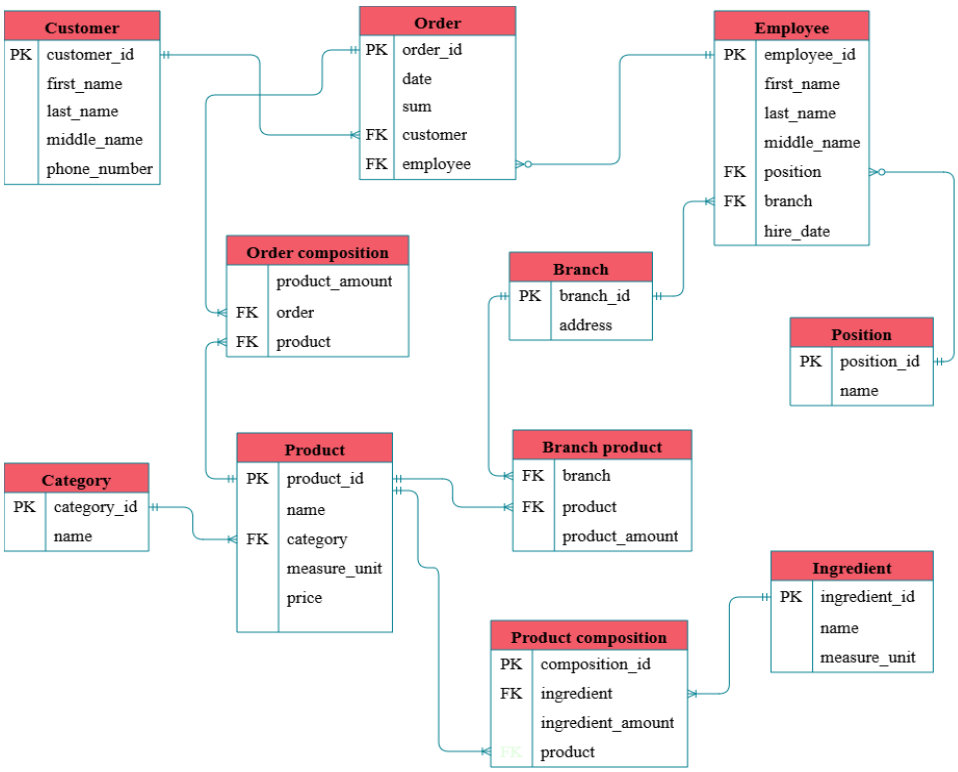

Рисунок 2 – ER-диаграмма

Для повышения надежности и удобства поддержки данных в модели использованы: пользовательский перечисляемый тип и пользовательский домен, описанные в таблице 2 и таблице 3 соответственно.

Таблица 2 – Таблица перечислений

Имя перечисления	**Описание**	**Множество значений**
measure_unit	Единица измерения товара/ингредиента	{‘шт’, ‘кг’, ‘г’, ‘мл’}

Таблица 3 – Таблица доменов

Имя домена	**Описание**	**Тип данных**	**Значение по умолчанию**	**Ограничения**
price	Денежный тип (цена)	DECIMAL(10,2) (дробь)	нет	> 0

Все связи между таблицами реализованы через внешние ключи. Они описаны в таблице 4.

Таблица 4 – Таблица описания связей

Поле первичного ключа	**Поле внешнего ключа**	**Политика управления ON UPDATE**	**Политика управления ON DELETE**
Customer.customer_id	Order.customer	CASCADE	RESTRICT
Order.order_id	Order_composition.order	CASCADE	RESTRICT
Employee.employee_id	Order.employee	CASCADE	RESTRICT
Position.position_id	Employee.position	CASCADE	RESTRICT
Branch.branch_id	Employee.branch	CASCADE	RESTRICT
Branch.branch_id	Branch_product.branch	CASCADE	RESTRICT
Product.product_id	Order_composition.product	CASCADE	RESTRICT
Product.product_id	Branch_product.product	CASCADE	RESTRICT
Category.category_id	Product.category	CASCADE	RESTRICT
Product.product_id	Product_ composition.product	CASCADE	RESTRICT
Ingredient.ingredient_id	Product_composition.ingredient	CASCADE	RESTRICT

Все атрибуты имеют четко определенные типы данных, обязательность, уникальность и ограничения. Они описаны в таблице 5.

Таблица 5 – Таблица атрибутов

Таблица	Наименование атрибута	Описание	Тип данных	Значение по умол-ча-нию	Обязательность	Уникальность	Ограничения	Первичный ключ	Внешний ключ
Customer	customer_id	Идентификатор клиента	INT	SERIAL	+	+	-	+	-
	first_name	Имя	VARCHAR(100)	-	+	-	-	-	-
	last_name	Фамилия	VARCHAR(100)	-	+	-	-	-	-
	middle_name	Отчество	VARCHAR(100)	-	-	-	-	-	-
	phone_number	Номер телефона	VARCHAR(100)	-	+	+	-	-	-

Order	order_id	Идентификатор заказа	INT	SERIAL	+	+	-	+	-
	date	Дата	DATE	-	+	+	-	-	-
	sum	Сумма заказа	price	-	+	-	> 0	-	-
	customer	Ссылка на покупателя	INT	-	+	-	-	-	+
	employee	Ссылка на продавца	INT	-	+	-	-	-	+
Employee	employee_id	Идентификатор сотрудника	INT	SERIAL	+	+	-	+	-
	first_name	Имя	VARCHAR(100)	-	+	-	-	-	-
	last_name	Фамилия	VARCHAR(100)	-	+	-	-	-	-
	middle_name	Отчество	VARCHAR(100)	-	-	-	-	-	-
	position	Ссылка на должность	INT	-	+	-	-	-	+

	branch	Ссылка на филиал	INT	-	+	-	-	-	+
	hire_date	Дата найма	DATE	-	+	-	-	-	-
Branch	branch_id	Идентификатор филиала	INT	SERIAL	+	+	-	+	-
	address	Адрес точки (филиала)	VARCHAR(100)	-	+	+	-	-	-
Position	position_id	Идентификатор должности	INT	SERIAL	+	+	-	+	-
	name	Название должности	VARCHAR(100)	-	+	+	-	-	-
Order_composition	product_amount	Количес-тво товара	DECIMAL(10,2) (дробь)	-	+	-	> 0	-	-
	order	Ссылка на заказ	INT (число)	-	+	-	-	+	+
	product	Ссылка на товар	INT (число)	-	+	-	-		+
Product	product_id	Идентификатор товара	INT	SERIAL	+	+	-	+	-

	name	Наименование	VARCHAR(100)	-	+	+	-	-	-
	category	Ссылка на категорию	INT (число)	-	+	-	-	-	+
	measure_unit	Единица измерения	measure_unit	-	+	-	-	-	-
	price	Стоимость товара	price	-	+	-	> 0	-	-
Category	category_id	Идентификатор категории	INT	SERIAL	+	+	-	+	-
	name	Наименование	VARCHAR(100)	-	+	+	-	-	-
Branch_product	branch	Ссылка на филиал	INT (число)	-	+	-	-	+	+
	Product	Ссылка на товар	INT (число)	-	+	-	-		+
	product_amount	Количество товара	DECIMAL(10,2) (дробь)	-	+	-	>= 0	-	-

Product_composition	composition_id	Идентификатор состава товара	INT	SERIAL	+	+	-	+	-
	ingredient	Ссылка на ингредиент	INT (число)	-	+	-	-	-	+
	ingredient_amount	Количество ингредиента	DECIMAL(10,2) (дробь)	-	+	-	> 0	-	-
	product	Ссылка на товар	INT (число)	-	+	-	-	-	+
Ingredient	ingredient_id	Идентификатор	INT	SERIAL	+	+	-	+	-
	name	Наименование	VARCHAR(100)	-	+	+	-	-	-
	measure_unit	Единица измерения	measure_unit	-	+	-	-	-	-

Таким образом, реляционная модель обеспечивает:

- полное соответствие предметной области;
- поддержку всех бизнес-процессов;
- защиту от аномалий обновления благодаря нормализации;
- основу для последующего физического проектирования в PostgreSQL.

4 ФИЗИЧЕСКОЕ ПРОЕКТИРОВАНИЕ

На этапе физического проектирования реляционная модель была реализована в СУБД PostgreSQL – одной из наиболее надежных и функциональных систем управления реляционными базами данных с открытым исходным кодом. Выбор PostgreSQL обусловлен его поддержкой:

- пользовательских типов данных (ENUM, DOMAIN);
- декларативных ограничений целостности;
- процедурного языка PL/pgSQL для написания триггеров и функций;
- гибкой системы ролей и управления привилегиями.

Реализация выполнена с полным соответствием логической модели и включает следующие компоненты:

1. Очистка и инициализация схемы.

Перед созданием объектов БД выполняется удаление уже существующих таблиц, доменов и типов данных, что позволяет обеспечить воспроизводимость скрипта при повторных запусках.

2. Пользовательские типы и домены, описанные в предыдущей главе.
3. Создание таблиц.

Все 11 таблиц создаются в соответствии с реляционной моделью. Для каждой таблицы первичные ключи объявлены через SERIAL PRIMARY KEY и внешние ключи определены с политиками:

- ON UPDATE CASCADE – автоматическое обновление зависимых записей при изменении первичного ключа;
- ON DELETE RESTRICT – запрет удаления записи, на которую есть ссылки, что предотвращает потерю целостности.

Особое внимание уделено составным первичным ключам:

- В таблице Order_composition первичный ключ – (order_id, product), что гарантирует, что один товар не может быть добавлен в один заказ дважды.

- В таблице Branch_product первичный ключ – (branch, product), что обеспечивает уникальность остатка товара в каждом филиале.

4. Наполнение тестовыми данными.

После создания структуры БД выполняется вставка небольшого набора тестовых данных, что позволит протестировать все функции приложения.

Таким образом, физическая реализация обеспечивает:

- точное соответствие предметной области и требованиям технического задания;
- надежное хранение данных за счет ограничений и типов PostgreSQL;
- готовность к интеграции с клиентским приложением;
- возможность дальнейшего масштабирования (добавление поставщиков, поставок, склада ингредиентов и т.п.).

Скрипт содержится в Приложении 1.

5 ОБЕСПЕЧЕНИЕ ЦЕЛОСТНОСТИ

Для поддержания логической согласованности данных и автоматизации ключевых бизнес-процессов в базе данных реализованы три пользовательские функции и три триггера, написанные на языке PL/pgSQL.

1. Пользовательские функции:
    - get_revenue_period(start_date DATE, end_date DATE) – вычисляет общую выручку за заданный период. Используется в финансовых отчетах администратора. При отсутствии заказов возвращает 0.
    - check_product_availability(order_id INT, product_id INT) – проверяет, достаточно ли указанного товара в том филиале, где работает сотрудник, оформивший заказ.
    - get_popular_products(start_date DATE DEFAULT NULL, end_date DATE DEFAULT NULL, top_count INT DEFAULT 10) – возвращает рейтинг самых продаваемых товаров за указанный период (или за все время, если даты не заданы). Для каждого товара возвращается: ID, название, общее количество продаж и выручка. Используется в отчете менеджера.
2. Триггеры:
    - tr_check_stock_availability (срабатывает до INSERT в Order_composition) – проверяет, что запрашиваемое количество товара не превышает текущий остаток в филиале, где работает продавец. Гарантирует, что невозможно оформить заказ на товар, которого нет в наличии.
    - tr_update_stock_on_order (срабатывает после INSERT в Order_composition) – автоматически списывает заказанное количество товара с остатков в соответствующем филиале. Обеспечивает актуальность данных в таблице Branch_product без участия приложения.
    - tr_update_order_sum (срабатывает после INSERT, UPDATE, DELETE в Order_composition) – пересчитывает итоговую сумму заказа как сумму произведений количества и цены для всех товаров в составе. Обновляет поле sum в таблице "Order". Гарантирует, что сумма заказа всегда соответствует его содержимому.

Все триггеры и функции используют транзакционную безопасность: при возникновении ошибки изменения автоматически откатываются. Это исключает возможность частичного обновления данных и сохраняет целостность учетной системы.

Скрипт содержится в Приложении 1.

6 ОБЕСПЕЧЕНИЕ БЕЗОПАСНОСТИ

Для защиты данных в системе применена ролевая модель доступа, реализованная средствами СУБД PostgreSQL. Доступ пользователей к данным разграничен в соответствии с их функциональными обязанностями, определенными в техническом задании.

В базе данных созданы четыре роли, соответствующие категориям пользователей:

- client_role – клиент;
- seller_role – продавец;
- manager_role – менеджер;
- admin_role – администратор.

Каждая роль получает минимально необходимый набор прав, достаточный для выполнения ее задач, но не позволяющий вмешиваться в данные других пользователей или нарушать целостность системы.

Права на объекты схемы распределены следующим образом:

- Права для роли client_role (клиент).

  SELECT на таблицы:

  - Product (просмотр ассортимента),
  - Category (категории товаров),
  - Branch (точки продаж).

  Клиент не имеет доступа к данным других пользователей, заказам, остаткам или персоналу.

- Права для роли seller_role (продавец):

  SELECT на таблицы:

  - Product, Category, Branch,
  - Customer (для поиска зарегистрированных клиентов).

  INSERT в таблицу Customer (регистрация новых клиентов).

  SELECT, INSERT в таблицу "Order" (оформление заказов).

  SELECT, INSERT в таблицу Order_composition (добавление товаров в заказ).

  Продавец не может напрямую изменять остатки – списание происходит автоматически через триггеры при оформлении заказа.

- Права для роли manager_role (менеджер):

Полный доступ (SELECT, INSERT, UPDATE, DELETE) к таблицам:

- Product,
- Ingredient,
- Product_composition (управление ассортиментом и рецептами);

SELECT, UPDATE на таблицу Branch_product (контроль и корректировка остатков);

SELECT на заказы и составы для формирования отчетов.

- Права для роли admin_role (администратор):

ALL PRIVILEGES на все таблицы схемы (полный доступ к данным);

Таким образом, система обеспечивает надежную защиту данных на уровне СУБД и соответствует требованиям к информационной безопасности.

Скрипт содержится в Приложении 1.

7 ОПИСАНИЕ ПРИЛОЖЕНИЯ

Для взаимодействия с базой данных было разработано консольное клиентское приложение на языке Python, обеспечивающее удобный и безопасный интерфейс для всех категорий пользователей.

Технологический стек:

- язык программирования: Python 3.10+;
- psycopg2 – библиотека для работы с БД, обеспечивает надежное подключение к PostgreSQL, поддержку транзакций и корректную обработку SQL-запросов;
- модуль getpass используется для скрытого ввода пароля;
- модуль datetime для валидации и форматирования дат при вводе пользователем;
- модульная структура с разделением логики.

Проект состоит из трех основных модулей:

- db.py – класс DBConnection, отвечающий за установку соединения с БД, выполнение запросов и обработку ошибок.
- main.py – главный исполняемый файл:
  - запрашивает у пользователя роль и пароль;
  - устанавливает подключение под указанной ролью;
  - определяет тип пользователя и запускает соответствующее меню.
- processes.py – содержит реализацию бизнес-процессов для каждой роли:
  - client_menu() – просмотр ассортимента;
  - seller_menu() – регистрация клиента, оформление заказа;
  - manager_menu() – управление товарами, просмотр отчетов;
  - admin_menu() – полные отчеты, управление персоналом.

Основные функции и сценарии работы:

1. Клиент (client_role):
   - просматривает категории товаров;
   - выбирает категорию и видит список доступных изделий с ценой и единицей измерения;

- для каждого товара может проверить наличие в филиалах (выводится адрес точки и количество товара).

2. Продавец (seller_role):
   - авторизуется по ФИО;
   - регистрирует клиента;
   - оформляет заказ:
     - выбирает существующего клиента или регистрирует нового;
     - добавляет товар из наличия в корзину;
     - после подтверждения заказа получает чек с перечнем товаров и общей стоимостью.
3. Менеджер (manager_role):
   - добавляет/изменяет товары, категории, ингредиенты;
   - настраивает состав продукции;
   - просматривает отчет «Топ популярных товаров» за указанный период;
   - проверяет и при необходимости корректирует остатки в филиалах.
4. Администратор (admin_role):
   - формирует финансовый отчет за период (общая выручка);
   - просматривает полную историю заказов конкретного клиента;
   - получает отчет по использованию ингредиентов (какие и в каком количестве использованы);
   - добавляет новых сотрудников (с выбором должности и филиала);
   - просматривает все данные системы.

Таким образом, приложение обеспечивает полноценный, безопасный и удобный интерфейс для работы с базой данных, полностью соответствующий требованиям курсовой работы и бизнес-процессам пекарни-кондитерской.

Код содержится в Приложении 2.

8 РУКОВОДСТВО ПО РАБОТЕ С ПРОГРАММОЙ

Для запуска и использования разработанной системы необходимо выполнить подготовительные шаги, описанные ниже, а затем следовать инструкциям в зависимости от роли пользователя.

1. Подготовка базы данных:

- установите PostgreSQL на локальный компьютер или сервер;
- создайте новую базу данных bakery_susu;
- выполните SQL-скрипт физического проектирования (раздел 4) для создания структуры и тестовых данных;
- выполните SQL-скрипт создания триггеров и функций (раздел 5);
- выполните SQL-скрипт создания ролей и назначения прав пользователям (раздел 6).

2. Подготовка клиентского приложения:

- убедитесь, что установлен Python 3.10 или новее;
- установите библиотеку psycopg2 для работы с PostgreSQL;
- разместите файлы проекта (main.py, db.py, processes.py) в одной папке;
- убедитесь, что в файле db.py указаны правильные параметры подключения к вашей СУБД (хост: localhost, порт: 5432, база данных: bakery_susu).

3. Запуск приложения и авторизация:

- запустите main.py;
- программа запросит:
    - имя пользователя – роль (client_role, seller_role, manager_role, admin_role);
    - пароль.

При успешном подключении откроется меню, соответствующее вашей роли. Вам будут доступны действия, описанные в разделе 7.

ЗАКЛЮЧЕНИЕ

В результате проделанной работы выполнены поставленные задачи, а именно: разработана структура реляционной базы данных для системы «Пекарня-кондитерская», а также спроектировано приложение для работы с ней.

Разработанная система является готовым решением для автоматизации учета в небольшой пекарне-кондитерской. Она легко поддерживается, расширяема (например, за счет добавления модуля поставок или расчета себестоимости) и может служить основой для дальнейшего развития – в том числе для создания веб-интерфейса или интеграции с кассовыми системами.

ЛИТЕРАТУРА

1. Комаров, В. Путеводитель по базам данных : руководство / В. Комаров ; под редакцией Е. В. Рогова. – Москва : ДМК Пресс, 2024. – 520 с. – ISBN 978-5-93700-287-7. https://e.lanbook.com/book/456743
2. Волк, В. К. Базы данных. Проектирование, программирование, управление и администрирование : учебник для вузов / В. К. Волк. – 5-е изд., стер. – Санкт-Петербург : Лань, 2025. – 244 с. – ISBN 978-5-507-53648-1. https://e.lanbook.com/book/493991

ПРИЛОЖЕНИЕ 1. Создание Базы данных

```
-- Раздел 4:
DROP TABLE IF EXISTS Order_composition CASCADE;
DROP TABLE IF EXISTS Branch_product CASCADE;
DROP TABLE IF EXISTS Product CASCADE;
DROP TABLE IF EXISTS Product_composition CASCADE;
DROP TABLE IF EXISTS "Order" CASCADE;
DROP TABLE IF EXISTS Employee CASCADE;
DROP TABLE IF EXISTS Customer CASCADE;
DROP TABLE IF EXISTS Branch CASCADE;
DROP TABLE IF EXISTS Position CASCADE;
DROP TABLE IF EXISTS Category CASCADE;
DROP TABLE IF EXISTS Ingredient CASCADE;
DROP DOMAIN IF EXISTS price CASCADE;
DROP TYPE IF EXISTS measure_unit CASCADE;
CREATE TYPE measure_unit AS ENUM ('шт', 'кг', 'г', 'мл');
CREATE DOMAIN price AS DECIMAL(10,2)
  CONSTRAINT positive_price CHECK (VALUE > 0);
CREATE TABLE IF NOT EXISTS Position (
    position_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE IF NOT EXISTS Branch (
    branch_id SERIAL PRIMARY KEY,
    address VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE IF NOT EXISTS Employee (
    employee_id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    position INT NOT NULL REFERENCES Position(position_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
    branch INT NOT NULL REFERENCES Branch(branch_id) ON UPDATE CASCADE ON DELETE
RESTRICT,
    hire_date DATE NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS Customer (
    customer_id SERIAL PRIMARY KEY,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100),
    phone_number VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE IF NOT EXISTS Category (
    category_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE
);
CREATE TABLE IF NOT EXISTS Ingredient (
    ingredient_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    measure_unit measure_unit NOT NULL
);
CREATE TABLE IF NOT EXISTS Product (
    product_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    category INT NOT NULL REFERENCES Category(category_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
    measure_unit measure_unit NOT NULL,
    price price NOT NULL
);
CREATE TABLE IF NOT EXISTS Product_composition (
    composition_id SERIAL PRIMARY KEY,
    product INT NOT NULL REFERENCES Product(product_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
    ingredient INT NOT NULL REFERENCES Ingredient(ingredient_id) ON UPDATE CASCADE
ON DELETE RESTRICT,
    ingredient_amount DECIMAL(10,2) NOT NULL CHECK (ingredient_amount > 0)
);
CREATE TABLE IF NOT EXISTS Branch_product (
    branch INT NOT NULL REFERENCES Branch(branch_id) ON UPDATE CASCADE ON DELETE
RESTRICT,
    product INT NOT NULL REFERENCES Product(product_id) ON UPDATE CASCADE ON
DELETE RESTRICT,
    product_amount DECIMAL(10,2) NOT NULL CHECK (product_amount >= 0),
    PRIMARY KEY (branch, product)
```

```
);

CREATE TABLE IF NOT EXISTS "Order" (

    order_id SERIAL PRIMARY KEY,

    date DATE NOT NULL,

    sum price NOT NULL,

    customer INT NOT NULL REFERENCES Customer(customer_id) ON UPDATE CASCADE ON
DELETE RESTRICT,

    employee INT NOT NULL REFERENCES Employee(employee_id) ON UPDATE CASCADE ON
DELETE RESTRICT

);

CREATE TABLE IF NOT EXISTS Order_composition (

    order_id INT NOT NULL REFERENCES "Order"(order_id) ON UPDATE CASCADE ON DELETE
RESTRICT,

    product INT NOT NULL REFERENCES Product(product_id) ON UPDATE CASCADE ON
DELETE RESTRICT,

    product_amount DECIMAL(10,2) NOT NULL CHECK (product_amount > 0),

    PRIMARY KEY (order_id, product)

);

-- Тестовые данные

INSERT INTO Position (name) VALUES

('Пекарь'), ('Кондитер'), ('Продавец'), ('Администратор'),('Менеджер');

INSERT INTO Branch (address) VALUES

('пр. Героя России Е.Н. Родионова, д. 6'), ('ул. Университетская Набережная, д.
46'), ('ул. 40-летия Победы, д. 9');

INSERT INTO Employee (first_name, last_name, middle_name, position, branch,
hire_date) VALUES

('Мария', 'Седова', 'Михайловна', 4, 1, '2024-01-01'), ('Иван', 'Иванов',
'Иванович', 3, 1, '2024-01-15'), ('Мария', 'Петрова', 'Сергеевна', 3, 2, '2024-03-
20'), ('Алексей', 'Сидоров', 'Владимирович', 3, 3, '2024-11-10'), ('Ольга',
'Кузнецова', 'Михайловна', 5, 3, '2024-02-28');

INSERT INTO Category (name) VALUES

('Хлеб'), ('Выпечка'), ('Кондитерские изделия');

INSERT INTO Ingredient (name, measure_unit) VALUES

('Мука', 'кг'), ('Сахар', 'кг'), ('Яйца', 'шт'), ('Молоко', 'мл'), ('Сливки',
'мл'), ('Вода', 'мл'), ('Масло сливочное', 'г'), ('Дрожжи', 'г'), ('Соль', 'г'),
('Ягоды', 'г'), ('Шоколад', 'г'), ('Фруктовый джем', 'г'), ('Ванилин', 'г');

INSERT INTO Product (name, category, measure_unit, price) VALUES

('Хлеб Злаковый', 1, 'шт', 45.00), ('Багет Французский', 1, 'шт', 65.00), ('Эклер
пломбирный', 3, 'шт', 85.00), ('Торт "Прага"', 3, 'кг', 1200.00), ('Пирожок с
клубникой', 2, 'шт', 97.00), ('Крендель с малиной', 2, 'шт', 120.00);

INSERT INTO Product_composition (product, ingredient, ingredient_amount) VALUES

(1, 1, 0.3), (1, 6, 600.0), (1, 7, 0.05), (1, 8, 0.02),(2, 1, 0.5), (2, 6, 700.0),
(2, 7, 0.02), (2, 8, 0.01),(3, 5, 50.0), (3, 4, 300.0), (3, 1, 0.2), (3, 3, 2),
```

```
(3, 2, 0.1), (3, 7, 200.0),(4, 11, 400.0), (4, 2, 0.1),(4, 7, 0.3), (4, 1, 0.2),
(4, 12, 200.0), (5, 1, 0.3), (5, 10, 250.0), (5, 2, 0.15), (5, 3, 2), (5, 7,
200.0), (5, 8, 0.02), (6, 1, 0.34), (6, 10, 150.0), (6, 2, 0.1), (6, 3, 2), (6, 7,
150.0), (6, 8, 50.0);

INSERT INTO Customer (first_name, last_name, middle_name, phone_number) VALUES

('Анна', 'Смирнова', 'Юрьевна', '79161234567'),('Дмитрий', 'Волков', 'Васильевич',
'79031234568'),('Екатерина', 'Морозова', 'Павловна', '79261234569');

INSERT INTO "Order" (date, sum, customer, employee) VALUES

('2025-12-01', 195.00, 1, 2),('2025-12-02', 290.00, 2, 3),('2025-12-03', 1200.00,
3, 4),('2025-12-04', 207.00, 1, 2),('2025-12-05', 701.00, 2, 3);

INSERT INTO Order_composition (order_id, product, product_amount) VALUES

(1, 1, 1), (1, 2, 1), (1, 3, 1),(2, 3, 2), (2, 6, 1),(3, 4, 1),(4, 2, 1), (4, 5,
1), (4, 1, 1),(5, 3, 2), (5, 6, 2),(5, 5, 3);

INSERT INTO Branch_product (branch, product, product_amount) VALUES

(1, 1, 20), (1, 2, 15), (1, 3, 10), (1, 4, 5), (1, 5, 8), (1, 6, 12), (2, 1, 25),
(2, 2, 18), (2, 3, 12), (2, 4, 3), (2, 5, 6), (2, 6, 10),(3, 1, 30), (3, 2, 20),
(3, 3, 15), (3, 4, 4), (3, 5, 10), (3, 6, 5);

-- Раздел 5: Функция 1

CREATE OR REPLACE FUNCTION get_revenue_period(

    start_date DATE,

    end_date DATE

) RETURNS DECIMAL(10,2) AS $$

DECLARE

    total_revenue DECIMAL(10,2);

BEGIN

    SELECT SUM(o.sum)

    INTO total_revenue

    FROM "Order" o

    WHERE o.date BETWEEN start_date AND end_date;

    RETURN COALESCE(total_revenue, 0);

END;

$$ LANGUAGE plpgsql;

-- Функция 2

CREATE OR REPLACE FUNCTION check_product_availability(

    p_order_id INT,

    p_product_id INT

) RETURNS BOOLEAN AS $$

DECLARE

    employee_branch INT;

    available_amount DECIMAL(10,2);
```

```
    ordered_amount DECIMAL(10,2);
BEGIN
    SELECT e.branch INTO employee_branch
    FROM "Order" o
    JOIN Employee e ON o.employee = e.employee_id
    WHERE o.order_id = p_order_id;
    SELECT COALESCE(bp.product_amount, 0) INTO available_amount
    FROM Branch_product bp
    WHERE bp.product = p_product_id AND bp.branch = employee_branch;
    SELECT COALESCE(oc.product_amount, 0) INTO ordered_amount
    FROM Order_composition oc
    WHERE oc.order_id = p_order_id AND oc.product = p_product_id;
    RETURN available_amount >= ordered_amount;
END;
$$ LANGUAGE plpgsql;
-- Функция 3
CREATE OR REPLACE FUNCTION get_popular_products(
    start_date DATE DEFAULT NULL,
    end_date DATE DEFAULT NULL,
    top_count INT DEFAULT 10
) RETURNS TABLE (
    product_id INT,
    product_name VARCHAR(100),
    total_sold DECIMAL(10,2),
    total_revenue DECIMAL(10,2)
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        p.product_id,
        p.name as product_name,
        SUM(oc.product_amount) as total_sold,
        SUM(oc.product_amount * p.price) as total_revenue
    FROM Order_composition oc
    JOIN Product p ON oc.product = p.product_id
    JOIN "Order" o ON oc.order_id = o.order_id
    WHERE (start_date IS NULL OR o.date >= start_date)
```

```
      AND (end_date IS NULL OR o.date <= end_date)
    GROUP BY p.product_id, p.name
    ORDER BY total_sold DESC
    LIMIT top_count;
END;
$$ LANGUAGE plpgsql;
-- Триггер 1
CREATE OR REPLACE FUNCTION update_stock_on_order()
RETURNS TRIGGER AS $$
DECLARE
    employee_branch INT;
BEGIN
    SELECT e.branch INTO employee_branch
    FROM "Order" o
    JOIN Employee e ON o.employee = e.employee_id
    WHERE o.order_id = NEW.order_id;
    UPDATE Branch_product
    SET product_amount = product_amount - NEW.product_amount
    WHERE product = NEW.product AND branch = employee_branch;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER tr_update_stock_on_order
AFTER INSERT ON Order_composition
FOR EACH ROW
EXECUTE FUNCTION update_stock_on_order();
-- Триггер 2
CREATE OR REPLACE FUNCTION update_order_sum()
RETURNS TRIGGER AS $$

DECLARE
    new_sum DECIMAL(10,2);
BEGIN
    SELECT SUM(oc.product_amount * p.price)
    INTO new_sum
    FROM Order_composition oc
    JOIN Product p ON oc.product = p.product_id
```

```
    WHERE oc.order_id = (
        CASE
            WHEN TG_OP = 'DELETE' THEN OLD.order_id
            ELSE NEW.order_id
        END
    );
    UPDATE "Order"
    SET sum = COALESCE(new_sum, 0)
    WHERE order_id = (
        CASE
            WHEN TG_OP = 'DELETE' THEN OLD.order_id
            ELSE NEW.order_id
        END
    );
    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER tr_update_order_sum
AFTER INSERT OR UPDATE OR DELETE ON Order_composition
FOR EACH ROW
EXECUTE FUNCTION update_order_sum();
-- Триггер 3
CREATE OR REPLACE FUNCTION check_stock_availability()
RETURNS TRIGGER AS $$
DECLARE
    employee_branch INT;
    available_amount DECIMAL(10,2);
BEGIN
    SELECT e.branch INTO employee_branch
    FROM "Order" o
    JOIN Employee e ON o.employee = e.employee_id
    WHERE o.order_id = NEW.order_id;
    SELECT COALESCE(bp.product_amount, 0) INTO available_amount
    FROM Branch_product bp
    WHERE bp.product = NEW.product AND bp.branch = employee_branch;
    IF available_amount < NEW.product_amount THEN
```

```
        RAISE EXCEPTION 'Недостаточно товара ID:% в филиале. Доступно: %, 
требуется: %',
            NEW.product, available_amount, NEW.product_amount;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER tr_check_stock_availability
BEFORE INSERT ON Order_composition
FOR EACH ROW
EXECUTE FUNCTION check_stock_availability();
-- Раздел 6: Роли
DROP ROLE IF EXISTS client_role;
DROP ROLE IF EXISTS seller_role;
DROP ROLE IF EXISTS manager_role;
DROP ROLE IF EXISTS admin_role;
CREATE ROLE client_role LOGIN PASSWORD "client123";
CREATE ROLE seller_role LOGIN PASSWORD "seller123";
CREATE ROLE manager_role LOGIN PASSWORD "manager123";
CREATE ROLE admin_role LOGIN PASSWORD "admin123";
-- Права: client_role
GRANT SELECT ON TABLE
      Product, Branch_product, Branch, Category
TO client_role;
-- seller_role
GRANT SELECT ON TABLE
      Product, Category, Branch_product, Branch, Employee, Position
TO seller_role;
GRANT SELECT, INSERT ON TABLE
      Customer, "Order", Order_composition
TO seller_role;
GRANT UPDATE ON TABLE
      "Order", Branch_product
TO seller_role;
GRANT USAGE ON SEQUENCE
    Customer_customer_id_seq, "Order_order_id_seq"
TO seller_role;
```

```
-- manager_role

GRANT SELECT ON TABLE

      Branch

TO manager_role;

GRANT SELECT, INSERT, UPDATE ON TABLE

      Customer, Order_composition, "Order", Branch_product

TO manager_role;

GRANT SELECT, INSERT, UPDATE, DELETE ON TABLE

      Category, Product, Product_composition, Ingredient, Employee, Position

TO manager_role;

GRANT USAGE ON SEQUENCE

    Customer_customer_id_seq, "Order_order_id_seq",     Product_product_id_seq,
Category_category_id_seq,     Branch_branch_id_seq, Employee_employee_id_seq,
Position_position_id_seq, Ingredient_ingredient_id_seq,
Product_composition_composition_id_seq

TO manager_role;

-- admin_role

GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;

GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO admin_role;

GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO admin_role;
```

ПРИЛОЖЕНИЕ 2. Код программы

Исходный код проекта приведен в электронном виде.